LÖSUNGEN UND MODERATION · WELT 6 · DS 38

W6·1 – Korrektur und Fehlerjagd: Lösungen

Lehrerexemplar · Lösungen der Fehlerjagd P1 – P10 (Java-Fassungen mit javac 21 und Nachbauten der Bibliotheksklassen geprüft) · Moderationshinweise zu Korrektur, Team- und Kursstatistik und Papier-Compiler

Moderationshinweise

Phase	Zeit	Hinweise
Vorbereitung	–	Probeklausuren vor DS 38 nur sichten (nicht korrigieren), auffällige Fälle notieren. Korrekturbogen = Kernpunkte des Erwartungshorizonts (identische Texte, BE und P-Nummern; Musterlösungs-Code bleibt beim Lehrerexemplar). Workspace „W6 Papier-Compiler“ mit dem Gerüst anlegen, Bibliothek „Niedersachsen“ aktivieren und austeilen.
1 Selbstkorrektur	20 min	Klausuren zurückgeben, rote Stifte. Regel betonen: Ein Kernpunkt zählt nur, wenn er da steht. Für d) und f) die Tabellen aus dem EH an die Tafel/den Beamer (Kernpunkte enthalten die entscheidenden Werte; vollständige Tabellen nur zeigen, nicht austeilen).
2 Peer-Check	15 min	Ring-Tausch im Team, eigene Häkchen abdecken. Lehrkraft geht herum und entscheidet Streitfälle mit dem Kernpunkt-Text; Stichproben (mind. eine Klausur je Team) selbst zweitkorrigieren. Es wird nichts gebucht – Punkte vergibt nur das Lab automatisch.
3 Team- und Kursstatistik	10 min	Striche je P-Nummer sammeln; Teamsummen an der Tafel als Kurssumme zusammenführen (Kurs-Top-3 für die Klausurvorbereitung) und mit dem Fallen-Profil des Kurses in der Lab-Auswertung vergleichen. Strichliste fotografieren – Vergleichsbasis für den Warm-up-Nachtest (DS 39).
4 Fehlerjagd	25 min	Teams starten mit ihren Top 3, verteilen den Rest. Kontrolle über dieses Lösungsblatt (am Pult oder Team-Expertin). Schnelle Teams: Warm-up-Karten W6 oder Lab-Level 6.1 und 6.2 (Fallenjagd).
5 Abi-Fahrplan	10 min	Einzelarbeit, bleibt beim Schüler. Auf konkrete Gegenmaßnahmen achten („Indexgrenze an den Rand schreiben“ statt „besser aufpassen“).
6 Papier-Compiler	Rest + zu Hause	Erster Lauf zählt: Ergebnis vor jeder Änderung notieren lassen (Screenshot oder Eintrag im AB). Hinweis: P5 (== statt equals) und eine Schleife statt Rekursion in h) erkennen die Tests nicht.

Stolpersteine

- Zu großzügige Selbstkorrektur („das meinte ich doch“) – deshalb Peer-Check mit abgedeckten Häkchen und Lehrer-Stichproben.
- Mehrere P-Nummern an einem Kernpunkt (z. B. „P2 P9“): Die Lernenden wählen die passende – nicht beide stricheln.
- Nicht erreichte Kernpunkte ohne P-Nummer (–) werden nicht gestrichelt (z. B. Hilfsschlange nicht erzeugt).

Lösungen der Fehlerjagd

P1 Absturz verhindern – Laufzeitfehler nicht ausgeschlossen

a) obersteRuecksendung() bei einer Station ohne Rücksendungen: top() auf leerem Stapel → IllegalStateException. fachnummerAn(7) für die Station Wiesenau (7 Fächer, Indizes 0 bis 6) bzw. fachnummerAn(−1): getItem mit ungültigem Index → IndexOutOfBoundsException (beides geprüft).

b) Geprüft: leere Station → "", Index 7 → -1, Index −1 → -1, Index 0 → 101, Index 6 → 107.

```
public String obersteRuecksendung() {
    if (ruecksendungen.isEmpty()) {
        return "";
    }
    Paket p = (Paket) ruecksendungen.top();
    return p.getNummer();
}

public int fachnummerAn(int index) {
    if (index < 0 || index >= faecher.getLength()) {
        return -1;
    }
    Fach f = (Fach) faecher.getItem(index);
    return f.getNummer();
}
```

P2 Der verschwundene Stapel – Struktur zerstört statt unverändert

a) Rückgabe 1 (nur P-7781 hat die Größe L); s ist danach **leer** (geprüft).

b) s ist ein Parameter, also eine lokale Variable, die auf den Stapel des Aufrufers verweist. `s = hilf;` lässt nur diese lokale Variable auf den Hilfsstapel zeigen – der Stapel des Aufrufers bleibt leer. Außerdem liegt in hilf alles in umgekehrter Reihenfolge (P-1190 oben).

c) Statt `s = hilf;` zurückschichten (geprüft: Rückgabe 1, s danach wieder P-2741, P-0412, P-7781, P-5236, P-1190 von oben):

```
while (!hilf.isEmpty()) {
    s.push(hilf.pop());
}
```

P3 Signatur-Check – Signatur verfehlt

a) (1) Rückgabetyp void statt boolean (Wahrheitswert) – return false ist dann ein Fehler; (2) Parameter String statt char (Zeichen) – der Vergleich `f.getGroesse() == groesse` vergleicht char mit String (Compilerfehler); (3) am Ende fehlt `return true;` – nicht jeder Pfad liefert einen Wert. (Beide Compilerfehler mit javac geprüft.)

b) Geprüft: Wiesenau → istVoll('S') = false, istVoll('M') = false; Station ohne Fächer → true.

```
public boolean istVoll(char groesse) {
    for (int i = 0; i < faecher.getLength(); i++) {
        Fach f = (Fach) faecher.getItem(i);
        if (f.istFrei() && f.getGroesse() == groesse) {
            return false;
        }
    }
    return true;
}
```

P4 Ganz schön ganzzahlig – Typen und Casts

a) Zeile `Paket p = auftraege.dequeue();` – dequeue liefert Object; ohne Cast (Paket) Compilerfehler (geprüft).

b) Volumen der Aufträge: 2 + 3 + 1 + 2 + 2 + 3 = 13 bei 6 Aufträgen. summe / anzahl ist eine ganzzahlige Division: 13 / 6 = 2, zurückgegeben wird 2.0. Richtig: 13 / 6 ≈ 2,17 (geprüft).

c) `return (double) summe / anzahl;` (oder `summe * 1.0 / anzahl`; falsch wäre `(double) (summe / anzahl)`).

P5 Gleich ist nicht gleich – Zeichenketten mit == statt equals

a) `f.getInhalt().getEmpfaenger() == name` → `f.getInhalt().getEmpfaenger().equals(name)`.

b) Die Online-IDE vergleicht Zeichenketten mit == inhaltlich, deshalb sind die Tests grün. In Java vergleicht == bei Objekten die Referenzen: Mit einem Literal "Brandt" liefert die Methode zufällig 1, mit einer zur Laufzeit erzeugten Zeichenkette (new String("Brandt")) aber 0 (mit javac geprüft). Die Ergänzenden Hinweise verlangen den Gleichheitsvergleich – im Abitur kostet == den Kernpunkt.

c) Erlaubt: length(), charAt(i), substring(a, b), contains(t), compareTo(t), equals(t). Nicht erlaubt: toUpperCase(), indexOf(c), split(" "), trim().

P6 Eins daneben – Off-by-one

a) i startet bei faecher.getLength() = 7; getItem(7) → IndexOutOfBoundsException, bevor ein Fach geprüft wird (geprüft).

b) Die Bedingung i > 0 beendet die Schleife, bevor Index 0 geprüft wird. Ist nur Fach 101 (Index 0) frei, wird es nie gefunden → -1 statt 101 (geprüft).

c) `for (int i = faecher.getLength() - 1; i >= 0; i--)` – geprüft: nur 101 frei → 101, Station Wiesenau → 107, Station ohne Fächer → -1.

P7 Tiefgang – Rekursion

a) Rekursionstabelle (instrumentierte Java-Fassung):

Rekursionstiefe	i	Fach an Index i	Rückgabe
0 (Erstaufruf)	0	101 P-7310 (S)	3
1	1	102 frei	2
2	2	103 P-2208 (M)	2
3	3	– (i = getLength(): Basisfall)	0

b) 4 Aufrufe, größte Rekursionstiefe 3 (der Basisfall-Aufruf mit i = 3 zählt mit). Ergebnis 3.

c) Ergebnis 1 statt 3 (geprüft): Der Basisfall greift schon bei i = 2 und liefert 0, das letzte Fach 103 mit P-2208 (Volumen 2) wird nie geprüft. Bei einer Station ohne Fächer käme es zu einem Laufzeitfehler (getItem(0)).

P8 Trace-Detektiv – Trace-/Dokumentationstabelle

a) (1) Zeile i = 2: pos muss 1 bleiben – die Bedingung ist tage[i] > max (7 > 7 ist falsch), nicht ≥. (2) Die Zeile für i = 3 (tage[3] = 2, keine Änderung) fehlt – jede Zeile ist ein Durchlauf. (3) Die Endzeile fehlt: Nach dem letzten Durchlauf wird i auf 5 erhöht, die Schleife endet (5 < 5 ist falsch).

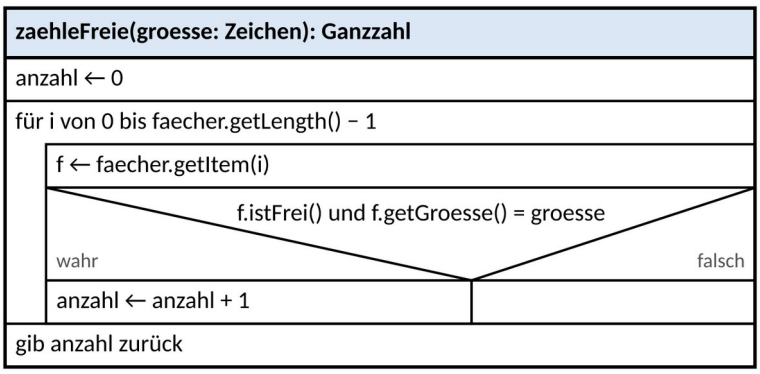
b) Richtige Tabelle (per Programm erzeugt):

i	tage[i]	max	pos
–	–	0	−1
0	3	3	0
1	7	7	1
2	7	7	1
3	2	7	1
4	9	9	4
5 (Ende)	–	9	4

P9 Notations-Polizei – Struktogramm nicht nach EH §5

a) Fehler: `anzahl = 0` statt `anzahl ← 0`; Java-Schleifenkopf `for (int i = 0; …; i++)` statt „für i von 0 bis faecher.getLength() – 1“; Typdeklaration `Fach f = …` statt `f ← …`; && statt „und“; == statt =; `anzahl++` statt `anzahl ← anzahl + 1`; `return anzahl;` (mit Semikolon) statt „gib anzahl zurück“.

b) Korrigiertes Struktogramm:



P10 Operator-Training – Operator verfehlt

a) A: **0 BE** – nur eine Nennung (FIFO) ohne Begründung und ohne Bezug zu den Zustellaufträgen. B: **1 BE** – beschreibt die Zugriffe, begründet aber nicht, warum gerade diese Reihenfolge für die Aufträge gefordert ist („besser“ ist kein Argument). C: **2 BE** – Anforderung aus dem Kontext (Reihenfolge des Eintreffens), Zugriffe der Schlange (dequeue/enqueue, FIFO) und Folgerung (kein Auftrag übergangen) sind verknüpft.

b) Beispiel: „Der Konstruktor Paketstation(standort, anzahlS, anzahlM, anzahlL) erzeugt eine Station und legt dabei gleich alle Fächer an: zuerst anzahlS Fächer der Größe S, dann anzahlM der Größe M und anzahlL der Größe L, fortlaufend ab 101 nummeriert und an die dynamische Reihung faecher angehängt. Für die Station Wiesenau (2, 3, 2) entstehen so die Fächer 101 bis 107 wie in Abbildung 2. Der Standort wird gesetzt, Schlange und Stapel sind zu Beginn leer.“